

AI Operating System Workshop

Day 3: AI Agents & AI Operating System

X-Company

2026

Section 1: AI Agent Architecture

An AI Agent is an autonomous system that perceives its environment, reasons about it, and takes actions to achieve goals. Unlike a simple LLM chatbot, an agent has persistent memory, access to tools, and can plan multi-step tasks.

Core Components

Component	Role	Examples
LLM (Brain)	Reasoning, planning, language understanding	LLaMA3, Mistral, GPT-4
Tools	Interact with external world	Search, calculator, file I/O, APIs
Memory	Store context across sessions	Vector DB, Redis, conversation history
Planning	Break goals into steps	ReAct, Chain-of-Thought, Tree-of-Thought

Agent Loop: Perceive → Think → Act

```
# Agent loop (simplified)
while not goal_achieved:
    # 1. PERCEIVE: observe current state
    observation = get_observation() # user input, tool result, etc.

    # 2. THINK: reason about next action
    thought = llm.think(observation, memory, available_tools)
    action = parse_action(thought) # e.g., {'tool': 'search', 'args': {'q':
'Python docs'}}

    # 3. ACT: execute chosen action
    result = execute_tool(action)
    memory.append({'observation': observation, 'action': action, 'result':
result})

    # Check if done
    goal_achieved = check_goal(result)
```

Section 2: Tool-Use & Function Calling

Ollama Function Calling Setup

```
import ollama

# Define tools
tools = [
    {
        'type': 'function',
        'function': {
            'name': 'calculator',
            'description': 'Performs arithmetic calculations',
            'parameters': {
                'type': 'object',
                'properties': {
                    'expression': {'type': 'string',
                                'description': 'Math expression to evaluate'}
                }
            }
        }
    },
    {
        'type': 'function',
        'function': {
            'name': 'search',
            'description': 'Searches the web for information',
            'parameters': {
                'type': 'object',
                'properties': {
                    'query': {'type': 'string',
                             'description': 'Search query'}
                }
            }
        }
    }
]
```

```

        'required': ['expression']
    }
}
}
]

response = ollama.chat(
    model='llama3.2',
    messages=[{'role': 'user', 'content': 'What is 15 * 24 + 7?'}],
    tools=tools
)

```

Example: Calculator + Search Tools

```

import math, requests

def calculator(expression: str) -> str:
    try:
        result = eval(expression, {'__builtins__': {}}, vars(math))
        return str(result)
    except Exception as e:
        return f'Error: {e}'

def web_search(query: str) -> str:
    # Example using DuckDuckGo
    resp = requests.get(f'https://api.duckduckgo.com/?q={query}&format=json')
    data = resp.json()
    return data.get('AbstractText', 'No results found')

TOOL_MAP = {'calculator': calculator, 'web_search': web_search}

def handle_tool_call(tool_call):
    fn = TOOL_MAP[tool_call['function']]['name']
    args = tool_call['function']['arguments']
    return fn(**args)

```

Section 3: RAG Agent

A RAG Agent combines the knowledge retrieval power of RAG with the planning and tool-use capabilities of an agent. The agent can decide WHEN to query the knowledge base, WHAT to search for, and HOW to combine results.

```

import ollama
from qdrant_client import QdrantClient

client = QdrantClient('localhost', port=6333)

def rag_tool(query: str) -> str:
    q_vec = ollama.embeddings(model='bge-m3', prompt=query)['embedding']
    results = client.search('workshop_docs', query_vector=q_vec, limit=5)
    return '\n'.join([r.payload['text'] for r in results])

tools = [{
    'type': 'function',
    'function': {
        'name': 'rag_tool',
        'description': 'Search the company knowledge base for relevant information',
        'parameters': {

```

```

        'type': 'object',
        'properties': {
            'query': {'type': 'string', 'description': 'Search query'}
        },
        'required': ['query']
    }
}
}}

TOOL_MAP = {'rag_tool': rag_tool}

# Agent loop with RAG
messages = [{'role': 'user', 'content': user_question}]
while True:
    response = ollama.chat(model='llama3.2', messages=messages, tools=tools)
    if response['message'].get('tool_calls'):
        for tc in response['message']['tool_calls']:
            result = handle_tool_call(tc)
            messages.append({'role': 'tool', 'content': result})
    else:
        print(response['message']['content'])
        break

```

Section 4: Multi-Agent Systems

Patterns Overview

Pattern	Description	Use Case	Example
Sequential	A → B → C (pipeline)	Data processing chains	Extract → Summarize → Translate
Parallel	A + B + C (concurrent)	Independent sub-tasks	Search multiple sources simultaneously
Routing	Router → A or B or C	Specialized agents	Classify query → route to specialist
Orchestrator	Manager → spawns workers	Complex long-horizon tasks	Project manager agent delegating

Orchestrator Pattern

```

class OrchestratorAgent:
    def __init__(self, workers: dict):
        self.workers = workers # {'research': ResearchAgent, 'write':
WriterAgent}

    def run(self, task: str) -> str:
        # 1. Plan the task
        plan = self.llm_plan(task)
        # plan = [{'agent': 'research', 'input': 'AI trends 2026'},
        #         {'agent': 'write', 'input': 'Draft report from research'}]

        context = {}
        for step in plan:
            agent_name = step['agent']
            agent_input = step['input'].format(**context)
            result = self.workers[agent_name].run(agent_input)
            context[f'{agent_name}_result'] = result

```

```
return context.get('write_result', 'Task complete')
```

Section 5: AI Operating System Concepts

An AI Operating System (AI OS) is a platform that coordinates multiple AI agents, skills, data sources, and automation schedules — much like a traditional OS coordinates processes, I/O, and scheduling for software programs.

Core Components

Component	Function	Analogy
Agents	Task executors with LLM + tools	OS processes
Skills	Reusable capability modules	OS libraries / drivers
Memory	Persistent state storage	File system + RAM
Heartbeat	Periodic background monitoring	OS cron / system daemon
Cron	Scheduled task execution	OS cron scheduler
Message Bus	Agent-to-agent communication	IPC / message queue

Architecture Diagram Description

The AI OS architecture consists of three layers:

- Infrastructure Layer: LLM servers (Ollama), Vector DB (Qdrant), Graph DB (Neo4j), Object storage
- Platform Layer: Agent runtime, Skill registry, Memory manager, Event bus, Scheduler
- Application Layer: Domain agents (HR, Finance, Sales), User interfaces (Telegram, Web), APIs

Section 6: Evaluation with RAGAS

RAGAS (Retrieval-Augmented Generation Assessment) provides automated metrics for evaluating RAG system quality without human labels.

4 Core Metrics

Metric	What It Measures	Formula	Target
Faithfulness	Is the answer grounded in context?	Claims supported / Total claims	> 0.8
Answer Relevancy	Does the answer address the question?	Cosine sim(answer, question)	> 0.8
Context Precision	Are retrieved chunks relevant?	Relevant chunks / Total retrieved	> 0.7
Context Recall	Are all key facts retrieved?	Key facts retrieved / Total key facts	> 0.7

Running RAGAS

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision,
context_recall
from datasets import Dataset

# Prepare evaluation dataset
data = {
```

```

    'question': ['What is the refund policy?'],
    'answer': ['Refunds are processed within 7 business days.'],
    'contexts': [['Customers may request refunds within 30 days...']],
    'ground_truth': ['Refunds take 7 business days to process.'],
}
dataset = Dataset.from_dict(data)

result = evaluate(
    dataset,
    metrics=[faithfulness, answer_relevancy, context_precision, context_recall]
)
print(result)

```

LLM-as-Judge Approach

Use an LLM to evaluate response quality when ground truth is unavailable:

```

judge_prompt = '''
Rate this RAG response on a scale of 1-5 for each criterion:
- Faithfulness: Is every claim supported by the context?
- Relevance: Does it answer the question?
- Completeness: Are all aspects of the question addressed?

Question: {question}
Context: {context}
Answer: {answer}

Return JSON: {{ "faithfulness": int, "relevance": int, "completeness": int,
"reasoning": str }}
'''

```

A/B Testing Methodology

- Define metric baseline (current system RAGAS scores)
- Implement variant B (different chunking, model, prompt, etc.)
- Run evaluation on same test set for both A and B
- Statistical significance: use t-test or bootstrap (n > 30 samples)
- Decision threshold: > 5% improvement on primary metric

Section 7: Capstone Project

Build an Enterprise AI Assistant

Project Goal: Build a production-ready AI assistant for your organization that integrates all workshop concepts.

Requirements

- Document ingestion: Support PDF, DOCX, with Thai language processing
- Hybrid search: Dense + BM25 with RRF fusion
- Knowledge graph: Entity extraction and Neo4j storage
- Multi-tool agent: At least 3 tools (RAG, search, calculator)
- Evaluation: RAGAS scores on 50-question test set
- UI: Telegram bot or simple web interface

Evaluation Criteria

Criterion	Weight	Minimum Score
Faithfulness (RAGAS)	25%	0.75
Answer Relevancy (RAGAS)	25%	0.75
Context Precision (RAGAS)	20%	0.70
Code quality & documentation	15%	Pass
Demo & presentation	15%	Pass

Section 8: Exercises & Reference

Exercise 1: Tool-Equipped Agent

Task: Build an agent with 3 tools: web search, calculator, and RAG. Handle tool chaining.

- Hint: Use while loop until no more tool_calls in response

Exercise 2: RAGAS Evaluation

Task: Create 20 Q&A pairs from your RAG system, evaluate with RAGAS, identify weak areas.

- Hint: Focus on Faithfulness — most RAG systems fail here first

Exercise 3: Multi-Agent Orchestration

Task: Build a 2-agent system: Research agent + Summary agent. Orchestrate them sequentially.

- Hint: Pass research_result as context to the summary agent

Reference

- Ollama Tools API: <https://ollama.ai/blog/tool-support>
- RAGAS: <https://docs.ragas.io>
- LangGraph (Multi-Agent): <https://langchain-ai.github.io/langgraph>
- OpenClaw (AI OS): <https://openclaw.ai>
- ReAct Paper: <https://arxiv.org/abs/2210.03629>